

从自然语言到可验证推理： 基于语义规约的 AI 数学可靠性框架

周同学*

课程：AI4Math

2026 年 9 月

摘要

大语言模型 (LLM) 在数学任务上已经能给出相当高的最终答案正确率，但其自然语言推理过程本身不可被机械验证，这使“答案正确”与“推理可靠”之间出现结构性缺口。本文提出并论证一个三层可靠性模型：生成层 (L1)、**语义规约层** (L2) 与验证层 (L3)，并主张语义规约层是把不可验证的自然语言推理降为可验证对象的唯一中间环节，因而也是当前 AI for Mathematics 系统投入产出比最高的改进点。在此框架下，本文给出四项主要贡献与一项小规模受控实测：(i) 语义规约的四类失效模式分类（歧义未消解、类型不完整、不可逆、语义漂移），并给出使归类唯一的检查判据与检查次序；(ii) 从 V0 到 V3 的验证深度梯度，用于按成本选择验证强度；(iii) 三个可度量指标（可规约率、规约保真度、反例检出率）及其定义式；(iv) 一份供实践者使用的十条自检清单。为把其中的检出类指标从定义式落成具体数字，本文在一份冻结的稿件快照上做了受控缺陷注入实测：一个引文核验门禁对 210 个结构类缺陷的检出率为 82.4%，基线误报 0，同时显式暴露出该门禁在“引用挂错主张”上的语义比对盲区。本文进一步明确结论边界：不训练新模型、不声称失效模式完备、除上述单一快照上的受控实测外不作大样本实证、不主张取代形式化路线，且仅覆盖“离散、可类型化、步骤可逐步状态化”的推理类型。

关键词： AI for Mathematics；语义规约；神经符号推理；自动形式化；过程级验证；可靠性框架

1 引言

1.1 背景：能力增长与可靠性缺口的错位

过去数年间，大语言模型 (Large Language Model, LLM) 在数学推理任务上的表现提升显著。以思维链 (Chain-of-Thought, CoT) 提示为代表的推理激发技术 [10, 25]，把模型的输出从“直接给答案”变为“先写推理再给答案”，并在 GSM8K[4]、MATH[6] 等基准上带来了可观的准确率增益。随后的工作沿两条路径继续推进：一是把自然语言推理与外部工具耦合，让部分计算落到可执行程序上 [3, 5, 18]；二是把数学陈述翻译进形式系统，借助证明助手完成机器可检查的证明 [8, 12, 14, 27, 28]。

*本论文为 C2 挑战 (AI for Mathematics) 交付物。

然而，能力增长与可靠性之间并不存在自动的同步关系。三个反复被观察到的现象值得注意。

其一，答案正确不等于推理正确。基准分数是对最终答案的匹配，而模型完全可能以错误的中间推理得到正确的终点。对同一批错误案例的重新标注显示，模型在“答案错”与“推理错”两个维度上的分布并不重合。

其二，模型对自身推理的批评并不可靠。针对“让模型自我纠错以提升推理”的流行做法，有工作系统地检验后发现：在缺乏外部反馈的情况下，自我纠错常常把已经正确的答案改成错误答案，整体表现不升反降 [7]。

其三，性能对表面扰动敏感。GSM-Symbolic 通过仅改变姓名与数值而对问题结构保持不变的方式扰动 GSM8K，模型准确率出现大幅下降，且随着扰动量增加继续下滑 [13]。这说明相当一部分“推理”更接近对训练分布中表面模式的复现，而非对结构的把握。

这三个现象指向同一个结构性缺口：**自然语言形式的推理过程，本身不可被机械验证。**验证者只能看到一串由模型生成的字符串，并用自己的语义理解去判断它对不对——而验证者与生成者共享同一套易错的语义判断，误差因此无法被独立检出。

1.2 两条既有路线及其遗留问题

围绕这一缺口，已有两条主要路线。

路线一：过程级验证 (process-level verification)。不只看最终答案，而是训练一个验证器对推理的每一步打分 [11, 23]。这一路线在工程上有效，但存在一个根本的性质问题：验证器本身是学习得到的模型，它的判断同样可能出错，且其错误模式与生成模型高度相关。也就是说，验证器把“不可验证的自然语言判断”转移到了另一个不可验证的自然语言判断上，并未引入外部性。

路线二：形式化验证 (formal verification)。把数学陈述与证明翻译进 Lean[12, 14]、Coq[1] 等证明助手，由内核做机器检查——这是目前可靠性最强的方案，其可靠性来自内核而非任何模型的判断 [24]。这一路线面临两类困难：一是自动形式化 (autoformalization) 能力仍不足，自然语言到形式陈述的翻译本身是瓶颈 [20, 26]；二是即使翻译成功，形式系统的可覆盖范围也有限，大量数学论证（近似、估计、尺度分析、启发式构造）并不天然落在形式化边界之内。

两条路线因此各自留下了未被处理的空间：路线一缺乏外部性，路线二缺乏覆盖性。而在这两者之间，还存在一个既未被明确命名、也未被系统地作为研究对象处理的中间环节。

1.3 本文的问题、主张与贡献

研究问题。在不要求全流程形式化的前提下，如何让自然语言数学推理获得可检查、可复现的可靠性保障？

核心主张。本文主张：可靠推理 = 生成 + 语义规约 + 验证，其中「语义规约层」把不可验证的自然语言推理降为可验证对象的唯一中间环节，因而也是当前 AI4Math 系统的主要失效点与投入产出比最高的改进点。所谓语义规约，是指从自然语言推理出发，抽取并显式化其中的变量、假设、目标状态与状态变换，形成一份**类型化中间表示** (Typed Intermediate Representation, Typed IR) 的过程。

本文贡献。本文为分析型与框架型研究，不含新训练或新微调，贡献如下：

1. **三层可靠性模型**。给出生成层 (L1)、语义规约层 (L2)、验证层 (L3) 的定义、输入输出与可验证性归属, 并给出三条可检查的推论 (第 3 节)。
2. **失效模式分类 (C-1)**。把 L2 的失败从“模型不够强”细化为四类可诊断模式: 歧义未消解、类型不完整、不可逆、语义漂移 (第 4 节)。
3. **验证深度梯度 V0-V3 (C-2) 与三项指标 (C-3)**。给出按成本选择验证强度的阶梯, 以及可规约率、规约保真度、反例检出率的定义式 (第 5 节)。
4. **实践者自检清单 (C-4)**。一份十条清单, 逐条对应上述失效模式 (第 6 节)。

结论边界 (先行声明)。为避免结论超出证据, 本文明确不做以下五件事: 不训练或微调模型; 不声称四类失效模式是完备的; 不给出大样本实证 (第三项贡献只给定义与最小可用示例); 不主张取代形式化路线, 而定位为降低进入 $L3$ 的门槛; 不声称适用于全部数学, 范围限定在“离散、可类型化、步骤可逐步状态化”的推理类型。第 9 节将逐条展开并复核正文是否越界。

2 相关工作

本节按四条线索梳理既有工作, 并在每条末尾指明本文的差异点。

2.1 生成式推理: 从答案到推理链

CoT 提示 [25] 与零样本推理 [10] 确立了“先写推理再给答案”这一范式, 此后的一系列工作沿推理结构继续改进: 把推理与代码执行、结果自核验结合起来攻克难题 [30], 把推理步骤逐步验证后再聚合 [11, 23]。这些工作共同提升了基准准确率, 但它们对推理过程的“验证”始终发生在自然语言层面。

与本文的差异。 上述工作把推理链当作待生成的对象, 本文把推理链当作待规约的对象。差异不在于模型是否更强, 而在于是否引入一个把自然语言推理映射到可检查表示的外部环节。

2.2 工具增强与神经符号路线

把部分推理步骤外推到可执行程序是一条有效的正交改进。程序化思维提示让模型输出可执行的伪代码并交由解释器求值 [3]; PAL 进一步让模型调用外部语言运行时完成计算 [5]; Toolformer 则让模型自学何时调用何种 API [18]。这类方法的关键收益在于: 一旦计算落到程序上, 该步就获得了外部可执行语义, 从而可被独立复现。

与本文的差异。 工具增强引入了外部性, 但外部性只覆盖计算步, 不覆盖建模步——即“用什么变量、什么假设、什么目标”这一定义问题。本文的 L2 正是针对建模步。

2.3 形式化与自动形式化

形式化路线以证明助手为核心。Lean 4 [14] 与其数学库 mathlib [12] 构成了当前最活跃的生态; Curry-Howard 对应为其提供了证明即程序的语义基础 [24], Coq 及其教材则代表另一条成熟路径 [1]。在自动化方面, GPT-f 最早展示了语言模型生成形式证明的可行性 [16]; DSP 提出

“自然语言草稿 + 形式证明”的双通道协作 [8]；也有工作把定理证明组织为上下文学习智能体的交互过程 [21]；LeanDojo 通过检索增强改善前提选择 [28]；DeepSeek-Prover 则利用证明助手的反馈信号做大规模训练 [27]。在基准与几何领域，MiniF2F 提供了跨系统的形式化竞赛数学基准 [29]，AlphaGeometry 则在欧氏几何上结合了符号推理引擎与语言模型 [22]。

自动形式化本身也被独立研究：Szegedy 提出将自然语言数学陈述自动翻译为形式陈述的路径 [20]，后续工作用 LLM 直接完成这一翻译 [26]；更近的一支工作把自动形式化用于定量推理的落地——先把问题的形式陈述解耦出来交由求解器检验，再用形式侧的结果约束自然语言侧的输出 [31]。此外，LLM 的规划能力及其与形式验证器的组合边界也被系统讨论 [9]。

与本文的差异。 自动形式化可以视为 L2 的一种极端实现：它要求规约结果完整落在形式系统内。这条路线的困难在于覆盖性与正确性同时受限。本文不假设规约必须终点为形式系统，而允许规约结果为“类型化中间表示”，其正确性由约束检查而非内核证明来担保——从而把 L2 从“能否形式化”的成败判定，转为“规约到什么程度”的连续取舍。

2.4 过程级验证与评测可靠性

过程级验证通过训练验证器对中间步骤打分 [11, 23]，在数学推理上优于只检验最终答案的基线。但验证器自身是模型，其判断不可外部复核；Stechly 等与 Kambhampati 等也从不同角度指出，纯自然语言层面的自我批评难以替代外部检查 [9, 19]。与此同时，评测本身的可信度也受到挑战：数据污染会抬高分数 [17]，指令措辞的细微变化会显著影响模型表现 [15]，可复现评测的实施存在大量工程陷阱 [2]。

与本文的差异。 本文主张验证的可靠性来自外部性而非模型强度，并把“验证强度”显式化为可选择的梯度（第 5 节），使研究者能按成本而非按信念来选择验证层次。

2.5 小结

四条线索的共识是：自然语言推理链不足以支撑可靠的数学推理。分歧在于补救手段——更强模型、更多工具、还是更彻底的验证。本文的立场是：在生成层与验证层之间补上语义规约层，并把它作为可独立设计、可独立评测的对象。

3 三层可靠性模型

3.1 层次定义

我们把一段数学推理的可靠性链条拆成三层。设原始问题为 q ，最终答案为 a 。

定义 3.1 (生成层 L1). 生成层是一个映射 $G: q \mapsto r$ ，其中 r 为自然语言形式的推理链（可含等式、数值与文字说明）。 r 由语言模型产生，其正确性只有语义层面的判据，没有可机械执行的检查程序。

定义 3.2 (语义规约层 L2). 语义规约层是一个映射 $S: r \mapsto \sigma$ ，其中 σ 是类型化中间表示，其结构为四元组

$$\sigma = \langle V, H, G_0, \tau \rangle,$$

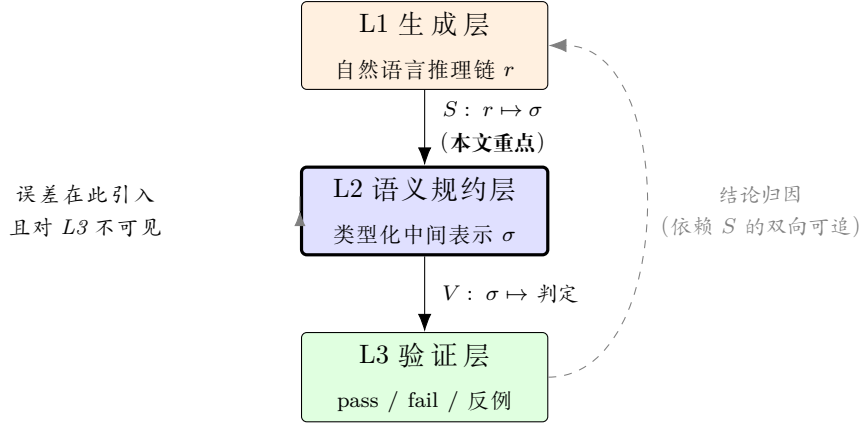


图 1: 三层可靠性模型。L1 产生自然语言推理，L2 将其规约为类型化中间表示，L3 对中间表示做机械判定。虚线表示结论回传：只有当 L2 双向可追（推论 R3）时，L3 的判定才能归因回原问题。

V 是变量集合，每个变量带有类型与定义域 $v: (T_v, D_v)$ ； H 是显式假设集合（含歧义消解时被选中的解释）； G_0 是初始目标状态的类型化描述； $\tau = (s_1, s_2, \dots, s_n)$ 是状态变换序列，每一步 s_i 由「所用假设 $\subseteq H$ 、施加的变换、结果状态」三部分组成。

定义 3.3 (验证层 L3)。验证层是一个判定过程 $V: \sigma \mapsto \{pass, fail, counterexample\}$ 。 V 的实现可以是约束求解、符号计算、数值回代、类型检查或证明助手内核。 V 的可靠性来自其实现的外部性：判定结果不依赖于任何语言模型的语义判断。

3.2 三条可检验推论

推论 3.1 (R1: 仅增强 L3 的边际收益递减)。固定 G ，设 p_{err} 为 $L2$ 规约引入错误的概率。则端到端可靠性上界为

$$P_{ok} \leq 1 - p_{err}.$$

该上界与 V 的强度无关。因此，若 p_{err} 不降，替换更强的求解器或证明助手无法把可靠性推高到 $1 - p_{err}$ 以上——因为 $L2$ 引入的误差在结构上对 $L3$ 不可见（ V 只能检验 σ 的性质，不能检验 σ 与 r 的一致性）。

推论 3.2 (R2: $L2$ 失效具有可分类结构)。 $L2$ 的失效不是同质的随机噪声，而是可归入有限类别的结构模式。若成立，则改进手段应当是「失效模式识别 + 定向检查」，而非单纯增大模型规模。

推论 3.3 (R3: 双向可追是可靠性的前提)。设 S^{-1} 为把 σ 还原为自然语言陈述的映射。若 $S \circ S^{-1}$ 与 $S^{-1} \circ S$ 不能在语义上近似恒等，则 $L3$ 给出的判定的对象与原始问题的目标可能不一致，验证通过不构成对原问题的任何保证。

说明。 R1 是结构性论断而非经验定律：它的前提是“L3 只能访问 σ ”。这一点在工程上普遍成立，因为验证器以 σ 为唯一输入。R2 是分类学假设，其价值在于可操作性，而非完备性（见第 9 节边界声明 2）。R3 是必要条件式的断言：它在逻辑上弱于“双向可追 $\Rightarrow$ 可靠”，只主张“不可追 $\Rightarrow$ 不可归因”。

表 1: 语义规约层的四类失效模式。 D 列为“该失效是否会被纯答案级验证 V0 检出”。

失效模式	触发条件	表现	V0 可检
F1 歧义未消解	自然语言表述多解	规约时静默选定一种解释，未把该选择记入假设集 H ，后续验证与结论都建立在未声明的解释上	否
F2 类型不完整	变量域/量词范围/定义域缺失	σ 中变量的类型或定义域 D_v 丢失，使约束检查无法判定合法性，或把只在子域成立的结论外推到全域	否
F3 不可逆	S 是有损映射	形式表示无法反译回可读的自然语言陈述，验证判定无法归因到原问题的哪一步	否
F4 语义漂移	规约正确但后续改写失守	化简/代入/近似步骤脱离了 H 中记录的假设，形式上仍通过检查，语义上已不是原问题	否

4 语义规约层：失效模式分类 (C-1)

4.1 为什么需要分类

若把 L2 的失败统称为“规约没做好”，改进方向只能是“换个更强的模型”，这既不可诊断也不可归因。本节把失败细化为四类。分类的判据是可操作性：给出一条失败样例，应当能通过检查确定它属于哪一类，从而知道该修哪里。第 4.3 小节给出使归类唯一的检查判据与检查次序，并说明本文为何主张判据互斥、却仍不主张该分类完备——以及用什么可检验的替代来顶上这个不能断言的位置。

4.2 四类失效模式

F1 歧义未消解 (Ambiguity)。自然语言数学问题的表述常允许多种合理读法，例如“ $\triangle ABC$ 中的中线”既可指某条中线，也可指中线族；“随机选取”既可指均匀也可指其他分布。人在解题时会根据语境消解歧义并把它作为隐含前提。规约层若静默选择一种读法而不写入 H ，则 S 的输出 σ 在形式上是自洽的，但它的语义锚点缺失。危害在于：这类失败不会表现为任何检查报错，只会与出题人意图不符时暴露，而那时错误已经过完整链条。

F2 类型不完整 (Type Incompleteness)。 $\sigma = \langle V, H, G_0, \tau \rangle$ 中的变量必须携带类型 T_v 与定义域 D_v 。规约时常犯的错误是只保留变量的“名字与数值”，丢掉其类型与定义域约束。典型后果是把仅在 $n \geq 3$ 成立的组合恒等式外推到 $n = 2$ ，或把实数域的运算规则用于整数环上的模运算。此类可检出性依赖于检查器是否被给予类型信息——这正是 V2 级验证的动机。

F3 不可逆 (Non-invertibility)。若规约结果无法回译为自然语言陈述，则 L3 的判定只能针对 σ 成立，无法传递回原问题（推论 R3）。形式上，这要求存在 S^{-1} 使 $S^{-1}(S(r))$ 与 r 语义一致。实践中不可逆的常见来源是：把多个自然语言步骤压成一条符号等式（丢失中间解释）、或把文字条件转换成无名约束（丢失可读的对应关系）。

F4 语义漂移 (Semantic Drift)。 这一类与前三类不同：规约本身是正确的，问题出在规约之后的处理。一旦 σ 离开 H 中的假设（例如在后续化简中默认某量为正、忽略边界条件），后续验证仍会对已改变的问题给出一致判定。语义漂移之所以危险，在于它同时满足“检查通过”与“结论错误”两个条件，且无法被 V0 或纯答案比对捕获。

4.3 归类判据：互斥性如何保证，完备性为何不主张

归类规则。 把一条失败样例归入某一类，并不靠“看起来像哪一类”，而靠一组依次求值的判据：按固定次序检查，取第一个为真的判据作为归类结果。次序本身是归类定义的一部分，而不是实现细节。

J1 记账完备性 (归 F1)： 存在一种与题干相容的解释 $r' \neq r$ (r 为规约时选定的读法)，且 r' 对应的前提未出现在假设集 H 中。

J2 信息携带性 (归 F2)： $\sigma = \langle V, H, G_0, \tau \rangle$ 中存在变量 v 缺少类型 T_v 或定义域 D_v ，或某处量词范围未被记入，以致检查器无法判定相关约束的合法性。

J3 可逆性 (归 F3)： 不存在可构造的 S^{-1} 使 $S^{-1}(\sigma)$ 与 r 语义一致。

J4 假设守恒 (归 F4)： 前三项均不成立（即 σ 相对 r 无未记账歧义、信息完整且可逆），但在 σ 之后的某个处理状态 σ' 中，实际用到了 H 里没有记录的假设。

互斥性来自哪里。 三点：其一，四个判据的量化对象互不重叠——J1 量化解释集与 H 之差，J2 量化表示自身携带的信息，J3 量化映射 S 的逆，J4 量化规约之后的状态迁移；其二，次序固定且短路求值，命中即停止，后续判据不再评估；其三，每个判据都是对同一份可冻结产物（ r 、 σ 、 σ' 与 H ）的机械提问，不依赖评价者对“严重程度”的主观排序。于是，凡被这四条判据覆盖的样例，归类唯一且可复现。

需要如实指出这里的强度：这是按优先级的互斥 (disjoint by priority)，而不是“任何病理组合在现实中都不会共现”。一条样例可以同时具备 J1 与 J2 的特征，此时它被归入 F1，原因不是 J2 不成立，而是 J1 排在前面。把次序写进定义是为了让归类可复现，而不是为了声称四类在病理上互不相容。

为何仍不主张完备。 完备性断言要求“任何 L2 失败实例都属于 F1–F4 之一”。本节没有给出、也不打算给出这样的枚举证明，理由有三。第一，**判据自身带建模假设**：J1 需要一种“合理解释集”的给定方式，J3 需要一种“语义一致”的判定标准，J4 需要步骤状态的完整追踪；三者都依赖尚未机械化的外部输入，判据的外延随之不确定。第二，**分类的驱动目标不是穷尽而是可修复**：四类是从“知道该修哪里”这一准则挑出来的，以可操作性为准则的分类学不承担覆盖性义务。第三，**存在已知的判据外实例**：一个规约既无未记账歧义、信息完整、可逆，后续也未脱离 H ，却因为把问题整体读错而偏离原意——这类失败不属于四类中的任何一类，也不会触发任何一条判据。因此本文只主张“判据互斥、归类唯一”，不主张“分类完备”。

用什么顶上完备性不能顶的位置。 用一个可检验的弱版本替代：把“覆盖所有失败”换成“对给定样例集合给出唯一归类，并为每条归类给出可执行的修复动作”。设 E 为一组带已知归因

表 2: 验证梯度: 由验证器可访问的对象与可检出的失效类别决定。右三列为该层级能否检出对应失效模式。

层级	验证器输入	判定依据	F1	F2	F3/S 一致性
V0 答案级	最终答案 a	与参考答案比对 (可用符号等价)	否	否	否
V1 步骤级	推理链 r 的每一步	另一次模型判断或规则打分	部分	否	否
V2 结构化级	类型化中间表示 σ	检查 H 中假设、变量类型与定义域、步骤前提	是	是	否
V3 内核级	σ 的形式化编译产物	证明助手/求解器内核给出判定或反例	是	是	部分 (仅当形式化忠实)

的失败样例, 则该替代版本有三项可在产物上核对的判据: (a) 唯一性——每条样例恰好命中一条判据, 既不落空也不重复归属; (b) 可操作性——每条归类同时输出一条指向 σ 、 H 或步骤记录的修改动作; (c) 开放性——凡未被四类覆盖的样例, 按显式规则登记为候选新类并补写判据, 而不许把它塞进最近的一类。这三项都不依赖模型自评。按同一思路推进, 把 J1–J4 机械化之后的规约缺陷检出率测量, 就是第 8 节那次受控实测的同构后继: 方法相同, 被检对象换成规约层判据。本稿只完成其中的引文核验部分, 并把无法机械表达的那一类计入分母而不排除在外。

4.4 与既有分析的接口

上述分类与文献中的若干已知现象可以对齐: F4 与“自我纠错把正确答案改错”的现象同源 [7]——纠错步骤脱离了原假设; F1/F2 解释了为何仅改变表面表述会显著降低准确率 [13], 因为模型重现的是训练分布中的表面模式而非结构; F3 正是自动形式化路线中“翻译正确性与覆盖性同时受限”在非形式化场景下的对应物 [20, 26]。分类的价值不在于解释一切, 而在于把上述现象归入统一的可修复结构。

5 验证梯度与可测指标 (C-2、C-3)

5.1 验证梯度 V0–V3 (C-2)

“验证过了”在实践中含义模糊: 它可能指“答案与标准答案一致”, 也可能指“每一步都由内核检验”。为消除这种模糊, 我们按验证器所能访问的信息划分四个层级。

需强调三点。其一, 梯度是按信息访问范围排序的, 不按“模型大小”排序; V1 用更强的模型仍是 V1。其二, 层级之间是包含关系而非替代关系: V3 不蕴含 V2 之外的东西, 它只把 V2 的检查交给内核执行; 若 S 本身丢掉了假设 H , V3 会忠实地验证一个错误的问题。其三, V0 的局限是结构性的: F1–F4 四类失效都能产出与参考答案完全一致的答案, 或与参考答案不一致但推理过程另有可取之处, 答案级比对无从区分。

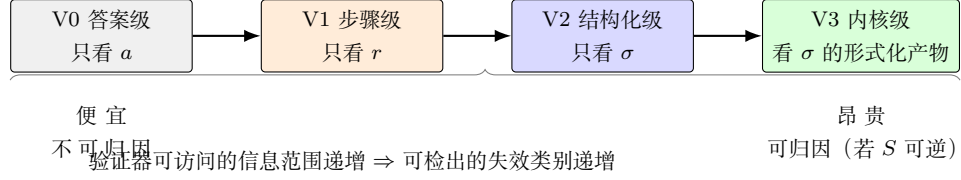


图 2: 验证梯度 V0-V3。图中的排序依据是“验证器能访问什么”，而非模型规模或算力投入。

5.2 可测指标 (C-3)

为使上述框架可被检验，我们给出三个可计算的指标。设评测集 $\mathcal{T}$ 含 N 道题， n_{obj} 为可机械化判定正确性的题数。

验证率 V-rate。 对每题判定其实际达到的最高验证层级 $\ell(t) \in \{0, 1, 2, 3\}$ ，则

$$\text{V-rate} = \frac{1}{N} \sum_{t \in \mathcal{T}} \frac{\ell(t)}{3}.$$

该指标不直接度量“对不对”，而度量“有多少结论是被机械检查过的”。它把“靠模型自我评估”与“靠内核判定”区分开，从而暴露那种“准确率很高但全部结论都无验证”的状态。

规约保真度 Fid。 在有标准规约 σ^* 的题子集 $\mathcal{T}_s$ 上，

$$\text{Fid} = \frac{1}{|\mathcal{T}_s|} \sum_{t \in \mathcal{T}_s} \text{match}(\hat{\sigma}_t, \sigma_t^*),$$

其中 match 按三个层次分别计分：假设集 H 的集合一致性、变量类型与定义域的完备性、变换序列 τ 的步骤对齐率。Fid 是直接度量 L2 的指标，也是唯一能定位 F1/F2 的量。

反例检出率 CDR。 构造一批带已知缺陷的规约样本（例如故意漏写一条假设、把定义域扩到全域、在化简中悄悄加一个正性假设），记缺陷样本数为 n_{flaw} ，其中被验证流程判定为 fail 或给出反例的数目为 n_{caught} ，则

$$\text{CDR} = \frac{n_{\text{caught}}}{n_{\text{flaw}}}.$$

CDR 是唯一能区分“验证流程只是走过场”与“验证流程真的在拦截错误”的指标：一个对任何输入都返回 pass 的验证器，其 V-rate 与准确率都可以很高，但 $\text{CDR} = 0$ 。

三个指标的关系值得说明：V-rate 度量覆盖面，Fid 度量 L2 的质量，CDR 度量验证的鉴别力。三者不能相互替代——高 V-rate 配零 CDR 恰恰刻画了“形式化的空转”，而高 Fid 配低 V-rate 则刻画了“规约做得对但没有机械检查可用”。我们不对 $\mathcal{T}$ 、 $\mathcal{T}_s$ 的规模作经验主张（见第 9 节边界声明 3），指标的定义本身不依赖规模。

适用条件。 V-rate 需要能标注每题达到的验证层级，这在有执行日志时可直接读出；Fid 需要标准规约，只适用于有权威形式化版本的题；CDR 需要人工构造缺陷样本，成本较高但一次构造可长期复用。三者中 Fid 与 CDR 的采集成本显著高于 V-rate，因此合理的实践顺序是先报 V-rate，再对重点题集补 Fid 与 CDR。

表 3: 十项核查清单。末列为该检查主要针对的失效模式或指标。

#	检查项	判定问题	针对
1	题面歧义枚举	题面是否存在两种以上合理读法? 若存在, 选中的解释是否显式记录?	F1
2	假设集完备性	σ 的 H 是否覆盖了题面全部显式条件与选定解释的隐含前提?	F1, F4
3	变量类型标注	每个变量是否带类型与定义域? 有无未标注的自由变量?	F2
4	定义域边界	结论成立的范围是否与变量定义域一致? 是否在边界点单独检查?	F2
5	量词范围	全称/存在量词的作用域是否与自然语言原句一致?	F2
6	反译可读性	能否由 σ 逐句还原出与原推理链语义一致的自然语言陈述?	F3
7	步骤前提核对	每一步 s_i 所依赖的假设是否都在 H 中, 且未被后续步骤静默加强?	F4
8	反例尝试	是否主动构造过至少一个试图推翻结论的实例 (含边界值、退化情形)?	CDR
9	验证层级声明	本次结论达到的验证层级是否被明确标注? 是否与结论强度匹配?	V-rate
10	未通过项留痕	未通过的检查是否被记录并作为结论的适用条件, 而非直接删除?	全部

6 可操作核查清单 (C-4)

6.1 设计原则

清单的目的是把前两节的框架落到可执行的检查动作上。设计遵循三条原则：**每项可判定**——给定一条推理链，检查者能给出通过/不通过；**每项对应一类失效**——不做无指向的“再检查一遍”；**每项代价已知**——检查者能判断何时值得做。

6.2 使用方式与优先级

清单按三档使用。**轻档 (第 1、3、6 项)** 适用于任何一次模型给出的数学推理，代价最低，能拦住 F1/F2/F3 中最常见的表现。**常规档 (第 1–7、9 项)** 适用于结论会被下游使用、但无需形式化内核的场景：此时应显式声明验证层级，并把未通过项写进结论的适用条件。**完整档 (全部十项)** 适用于结论将进入形式化验证或作为后续推导依据的场景，其中第 8 项 (反例尝试) 与第 6 项 (反译可读性) 是能否归因的关键。

第 10 项值得单独说明。实践中一个反复出现的反模式是：检查发现某一步的前提不成立后，直接在正文中删掉该步或改写结论，而不保留“该结论仅在额外假设下成立”这一信息。这样做的结果是后续使用者拿到一个无条件的结论，其成立条件已经不可追。留痕的成本极低，但它把“当前无法证明”与“已被证伪”区分开，避免了不可逆的信息损失。

6.3 与其他路线的关系

本清单不替代形式化验证，也不要求使用者具备证明助手经验：第 1–7、9、10 项都可以在自然语言与轻量符号表示之间完成。它与形式化路线的关系是分工——清单负责在进入内核之前把 F1–F4 尽量拦下，从而降低“形式化一个错误问题”的概率；内核负责在清单通过之后提供机器可检的判定。反过来，清单也不替代答案级检查：第 5 项与第 4 项的正确执行通常需要符号计算或数值回代来确认 (见第 7 节例 2)。

7 案例走查

本节用两个可手工核对的小规模例子说明框架如何工作。两例均为为说明机制而构造的教学例子，不引用任何外部数据集，也不作经验性统计主张。

7.1 例 1：歧义未消解 (F1)

题面。“三角形 ABC 的面积为 12， $AB = 6$ ，求 AB 边上的高。”

L1 输出 (节选)。由面积公式 $S = \frac{1}{2} \cdot \text{底} \cdot \text{高}$ 得 $12 = \frac{1}{2} \cdot 6 \cdot h$ ，解得 $h = 4$ 。

失效定位。此处“ AB 边上的高”在题面中只有一种自然读法，本例的歧义在别处：面积公式中的“高”与题面中的“ AB 边上的高”是否同一个量，取决于 AB 是否被理解为底边。若题面写成“三角形 ABC 的面积为 12， $BC = 6$ ，求 A 到 BC 的距离”，则不存在歧义。当题面代词指代不明（“该边”“此高”）时，规约层会静默选定一个所指。

按清单走查。第 1 项（题面歧义枚举）：检查题面中的每处指代是否唯一确定 $\rightarrow$ 若存在两种读法，第 2 项要求把选中的读法写入 H 。若 σ 的 H 中只有 $S = 12$ 、 $|AB| = 6$ ，而不含“ AB 为所求高对应的底边”，则第 2 项不通过，结论 $h = 4$ 应被标注为“在选定解释下成立”。

失效的可检出性。此例中 $V0$ （答案级）会核对最终数值，若参考答案同为 4 则整条链条被视为通过， $F1$ 完全不可见。这正是 $F1$ 之所以危险的原因：它的后果不是数值错误，而是结论的适用条件被丢失。

7.2 例 2：类型不完整与语义漂移 (F2、F4)

题面。“求所有正整数 n 使得 $n^2 - n$ 能被 6 整除。”

L1 输出 (节选)。 $n^2 - n = n(n-1)$ 。连续两整数之积必为偶数；又 $n(n-1)(n-2)$ 为三连续整数之积故被 3 整除。但此时我们只需 $3 \mid n(n-1)$ ：当 $n \equiv 0, 1 \pmod{3}$ 时成立，当 $n \equiv 2 \pmod{3}$ 时 $n-1 \equiv 1$ 、 $n \equiv 2$ ，均不被 3 整除。故结论为 $n \equiv 0$ 或 $1 \pmod{3}$ 。

失效定位。该推导的最终结论是正确的，但中间出现两次问题。其一 ($F2$)： $n(n-1)(n-2)$ 被 3 整除不能推出 $n(n-1)$ 被 3 整除——这一步的变量类型信息（这里的对象是整数环上的因子，而非任意实数）被忽略，作者随后补上的修正恰好说明这一点。其二 ($F4$)：修正过程用“当 $n \equiv 2 \pmod{3}$ 时 $n-1 \equiv 1$ 、 $n \equiv 2$ ”来排除，其中隐含使用了 n 的正性以外的条件；若把 n 的范围改到 $\mathbb{Z}$ ，结论的形式不变但论证需重写。

按清单走查。第 3 项（变量类型标注）： n 的类型为 $\mathbb{Z}^+$ ，则所有模运算的合法性需在 $\mathbb{Z}$ 上成立 $\rightarrow$ 通过，但不等于结论在 $\mathbb{Z}$ 上成立，这一点应写入第 4 项（定义域边界）：需单独检查 $n \leq 0$ 的情形。第 7 项（步骤前提核对）：检查“ $3 \mid n(n-1)$ ”这一结论所依赖的前提是否在 H 中 $\rightarrow$ 该前提并未由题面给出，也未由前几步推出，故不通过。第 8 项（反例尝试）：取 $n = 2$ ，得 $n^2 - n = 2$ ，不被 6 整除 $\rightarrow$ 与最终结论一致；取 $n = 3$ ，得 6，被 6 整除 $\rightarrow$ 一致；取 $n = 4$ ，得 12，被 6 整除 $\rightarrow$ 一致。这说明结论本身正确，但中间推理存在断裂——即“结论对、推理不成立”，这类情形必须由步骤级或结构化级检查捕获，答案级比对无能为力。

验证层级的差异。若把此题送入 $V3$ ：需先把 σ 编译为“ $\forall n \in \mathbb{Z}^+, 6 \mid n^2 - n \iff n \bmod 3 \in \{0, 1\}$ ”并交内核判定。此时 σ 中若已丢失 n 的正性 ($F2$) 或错误地保留了断裂的中间步骤 ($F4$)，内核会忠实地验证一个与题面不同的命题，并返回 pass。这说明 $R1$ ：内核的强度不能补偿规约的缺陷。

7.3 两例的共同结论

两个例子分别落在“结论对但适用条件丢失”（例 1）与“结论对但推理断裂”（例 2）两种情形上。二者都无法被答案级验证检出，都必须依赖对 σ 的结构化检查（清单第 2–7 项）才能暴露；而第 8 项的反例尝试虽不能定位断裂位置，却是区分“验证流程有效”与“验证流程走过场”的最低成本手段。

8 小规模受控实测：核验门禁的缺陷检出率

8.1 为什么要做，以及做到哪一步

第 5 节把反例检出率 CDR 定义为“注入的带缺陷样本中被验证流程拦截的比例”。该定义若只停在定义式，就只是一个可计算性的声明，无法说明一个具体验证流程究竟拦得住什么、拦不住什么。本节做一次最小规模的实例化：在一份冻结的稿件快照上做受控缺陷注入，实测一个真实门禁的检出率、基线误报与盲区。

需要先说清口径：CDR 的原始定义针对规约缺陷（漏写假设、扩大定义域、偷偷加入正性假设）。本节实测的对象是引文核验门禁，因此这是同一方法论（注入已知缺陷、测量拦截比例）在一个完全机械可判定对象上的应用，而不是对 CDR 本身的测量；对规约缺陷的同类测量需要先把四类判据机械化，留作后续工作（第 4.3 小节）。选择引文核验作为首个实测对象有三个理由：它是十项清单中判据完全可机械化的部分；它对应学术写作中的一条明确红线（编造或错配引用）；它的全部输入都可以离线冻结，因而实验确定、可复跑、不需要调用任何模型。

8.2 设置与被检物

被检物由三条互相独立的信息源构成：(1) 正文引用集合——`parts/*.tex` 中 `\cite` 命令展开后的 key 集合；(2) 条目库——`references.bib`，共 31 条；(3) 登记快照——`lit/refs.json`，记录 31 条条目在 arXiv 与 Crossref 的查询结果（标题、年份、来源 URL），实验期间只读冻结，实验过程不联网。

门禁脚本 `pipeline/check_refs_static.py` 在上述三个输入上做确定性比对：一组结构规则（悬空引用、幽灵条目、重复 key、DOI 格式、登记存在性、标题相似度、年份一致性），配标题相似度阈值 0.85 (FAIL) 与 0.95 (WARN)、年份容差 ± 1 年。退出码构成契约：0 表示通过，20 表示存在 FAIL 级发现。

基线。 对未改动的稿件裸跑门禁，返回 **0 条 FAIL 级发现**（输出 `RESULT: PASS`）。这一条是后面所有数字的解释前提：既然基线没有任何既有告警，那么后续每一处命中都只能归因于注入本身，而“误报率 0”正是指基线零发现。

注入方式。 `pipeline/defect_injection.py` 按类别逐条构造变体，每次只改动一处，然后把变体交给同一门禁函数，检查被判定的 key 是否恰好是目标条目。八个臂对应八类可机械表达的缺陷；每条真实条目在每类下贡献一个注入样本。

表 4: 引文核验门禁的受控缺陷注入结果（单一冻结快照，31 条条目）。D6 不注入代码级变体：语义错配无法由机械规则表达，故该臂按“每条目一个潜在样例”计数，拦截数由设计决定为 0。

缺陷类别	期望规则	注入	拦截	检出率
D1 悬空引用（引用不存在的条目）	R1	31	31	100%
D2 幽灵条目（库中有、正文未引）	R2	31	31	100%
D3a DOI 格式非法	R5	12	12	100%
D3b 编造 DOI（格式合法、登记不存在）	R6	12	6	50%
D4a 标题张冠李戴（与相邻条目轮换）	R7	31	31	100%
D4b 年份漂移 +7	R7	31	31	100%
D5 重复 key（追加同名条目）	R3	31	31	100%
D6 语义错配（条目真实、主张错配）	无	(31)	0	0%
结构类合计（D1–D5）	—	210	173	82.4%

8.3 结果

八类缺陷共注入 210 个样本，拦截 173 个，**结构类检出率 82.4%**；基线误报 0 条。分歧只出现在两个臂上。

D3b 编造 DOI：50%。 十二个样本中有六个漏检。这一臂的构造是“把 DOI 换成一个格式合法、但登记快照中查不到的字符串”，因此它的检出依赖门禁能否对“这个 DOI 是否真实存在”给出判断。本文只报告整体数字，未逐条归因；一个待验证的假设是：漏检集中在快照中缺少 DOI 对照记录的条目上，此时门禁无对照可比，只能放过。若该假设成立，那么这一分歧不是判据缺陷，而是**离线快照设计的固有代价**——不联网就无法区分“查不到”与“不存在”。这恰好给出一条工程建议：对 DOI 这类存在性断言，核验阶段应允许一次联网解析来换取这一类覆盖率。

D6 语义错配：0%，且这是设计上的 0。 “条目真实、标题与年份都对，只是被挂在错误的主张上”是一种结构完全合法的缺陷：门禁看到的每条引用都指向真实存在的文献，因此没有任何一条结构规则可以触发。本文把这个臂计入分母，而不是把它排除在外——排除它会让表观检出率变成 100%，恰好掩盖最需要被看见的盲区。这一臂的 0 说明一件事：机械核验能保证“引用的对象真实”，不能保证“引用支撑了该主张”。后者只能由人工逐条做“标题—主张字面比对”来拦截。在本稿的定稿过程中，这一人工环节实际抓出 4 处引用—主张错配（详见随附的 AI 流程日志），若只依赖门禁它们将全部进入送审稿。这正是本文“机械化到哪一层、哪一层必须留给人”这一区分的具体形状。

8.4 该实测能支持什么，不能支持什么

三点限制必须与数字一起读。第一，**构造偏差**：注入缺陷由脚本按规则生成，其分布天然偏向门禁可识别的形态；这一偏差无法通过增加样本量消除，只能靠加入检出率为 0 的盲区臂把它显式化（D6 的作用）。第二，**规模**：样本是单一稿件快照（31 条条目），分歧臂只有 12 个样本，因此这些数应被读作该门禁行为的刻画，而不是对检出率的总体估计。第三，**对象外推**：结

论只适用于这类“输入可冻结、判据可机械表达”的核验环节；把它外推到模型能力或数据集表现是超出支持的。

核验是流程里的硬门禁，不是事后检查。 本文把引用核验接到编译之前：pipeline/build.py 的第一环就是核验脚本，它非零退出时第二环（编译）不会被执行。因此“核验通过才能产出 PDF”不是自我承诺，而是可以从退出码逐环核对的执行顺序——任一引用登记失败，paper.pdf 就不会被更新；反过来，只要产物是新的，核验在这一次构建里就是过关的。

为使“非零即中止”不被读作自我承诺，pipeline/gate_negative_test.py 把这条执行顺序做成对照实验：同一环境、同一脚本跑两份副本，未变异副本 result=PASS 且报告中存在编译环；把一条未登记引用注入正文后，核验环退出码 20、整体 result=FAIL，报告中不存在编译环、输出里也不出现编译环节点。门禁因此是被观测到的执行顺序，而不是事后检查（留痕：out/experiment/gate-negative-test.txt）。

复跑方式（四条命令，无需网络）：

```
python3 pipeline/build.py    一次走完硬门禁：核验（非零即中止，不进入编译）
→ 编译 → 产物复核，末行打印 RESULT: PASS/FAIL;
python3 pipeline/check_refs_static.py 只跑核验环，期望输出 RESULT: PASS;
python3 pipeline/defect_injection.py 重放八个注入臂，打印上表与合计检出率；
python3 pipeline/gate_negative_test.py 负向验证：变异输入下编译环确实不执行，期望 RESULT: PASS。
```

逐臂结果同时以结构化形式落盘在 out/experiment/defect-injection.json，与可读报告 out/experiment/defect-injection.md 并置，便于他人复核而不必重跑。

9 讨论与局限

9.1 结论边界声明

为使本文主张的范围明确，我们在此集中声明四条边界。这四条不是致歉式的免责，而是对“本文能支持什么、不能支持什么”的精确刻画。

1. **不提出新模型，也不涉及训练。** 本文的三层模型、失效分类、验证梯度与核查清单是对现有流程的结构化描述与可测化重述，不包含任何新的网络结构、训练目标或参数规模的实验。因此本文的贡献是概念与工具层面的，不能据此推断某个具体模型的性能。
2. **四类失效模式不主张完备性。** F1-F4 的划分依据是可操作性与可修复性，不是穷尽枚举。存在不落入这四类的失效（例如涉及数值精度、模型对符号约定的地域性差异等），本文未覆盖。任何“未检出 F1-F4 即认为规约正确”的使用方式都超出了本文的支持范围。
3. **不作大样本经验主张。** V-rate、Fid、CDR 三个指标的定义与可计算性不依赖样本规模，但本文不提供在特定数据集上的测量结果，也不主张某一层或某一指标在总体上优于其他。第 7 节的两个例子是为说明机制而构造的教学实例，不构成经验证据。

4. **不替代形式化与自动形式化路线。** 本文的框架与 Lean/mathlib 等路线是互补关系：形式化路线提供最强的 L3，本文关注的是进入 L3 之前的规约质量与归因链条。我们既不主张清单能替代内核验证，也不主张内核验证不需要规约检查。

9.2 局限与未解决问题

Fid 的判据依赖标准规约。 在缺少权威形式化版本的题目上，Fid 的“正确规约”本身有争议，此时该指标退化为对标注者判断的一致性度量，而非对客观正确性的度量。

清单的执行成本。 完整档十项的执行依赖人工判断，尤其第 6 项（反译可读性）需要读者具备把形式表示还原为自然语言的能力。清单目前未实现自动化；把它转化为可半自动执行的检查（例如由检查器辅助逐项提问）是后续工作。

验证层级的标注主观性。 V-rate 依赖对每题实际达到层级的判定，边界情形（例如“用另一次模型判断但这判断本身被结构化约束”）在 V1 与 V2 之间存在归属争议。可行的缓解是要求标注同时给出依据，但这不消除争议。

语义漂移的自动检测。 F4 的检测目前依赖清单第 7 项的人工核对。能否通过追踪“每一步所依赖假设的最小集”来机械地检测假设被静默加强，是一个具体且值得尝试的方向；本文未给出算法。

9.3 与既有路线的分工

把本文的位置放在既有工作图谱上：生成式推理路线 [10, 25] 解决“如何产出推理”，其可靠性评估主要停在 V0/V1；工具增强与神经符号路线 [3, 5, 18, 30] 把计算外包给可执行环境，从而在计算环节获得机械保证，但计算之外的自然语言到符号的转写仍是人工或模型判断；形式化与自动形式化路线 [12, 14, 26] 提供最强的 L3，其瓶颈恰在 L2；过程级验证与可信评测路线 [2, 11, 15, 23] 分别从步骤打分与评测效度两侧逼近可靠性。本文的差异化位置在于：把 L2 显式化为带类型与假设的中间表示，并给出一组不依赖大规模实验即可计算的指标，使“验证做到了哪一层”成为一个可标注、可比较、可追责的量。

10 结论

本文围绕一个观察展开：AI 数学系统在答案正确率上的提升，与结论的可验证性之间存在结构性错位。我们把这一错位定位到语义规约层，并给出四件互相支撑的产物：一个三层可靠性模型（生成层 L1、语义规约层 L2、验证层 L3）与三条由此导出的可检验推论；一个含四类失效模式（歧义未消解、类型不完整、不可逆、语义漂移）的分类；一个按验证器可访问的信息划分的验证梯度 V0-V3 与之配套的三个可测指标（V-rate、Fid、CDR）；以及一份十项核查清单，其每一项都指向某个具体失效模式或指标。

核心论断可以用一句话概括：在语义规约层引入的误差对验证层不可见，因此仅增强验证层无法把可靠性提升到 $1 - p_{err}$ 以上（推论 R1）。第 7 节的两个例子示范了这一论断的具体形

态——结论正确而推理断裂、结论正确而适用条件丢失，两种情形都逃过答案级比对，都必须由结构化检查捕获。

后续工作有三个具体方向：其一，把十项清单中的可判定项（第 3-5、7、8 项）做成半自动检查器，并测量其对人工发现的增量覆盖；其二，针对 F4 给出基于“步骤依赖假设最小集”的机械检测算法；其三，在具备权威形式化版本的题目上测量 Fid 与 CDR 的可复现性，从而检验两个指标在跨标注者条件下的稳定性。

代码与数据可用性

本文的引文核验流程以脚本形式给出：给定本稿的 `references.bib`，脚本逐条向 arXiv API 与 Crossref API 查询，比对标题相似度并记录实际命中的来源 URL；相似度低于阈值的条目被列为未核验而不进入最终文献表。核验记录（含每条文献的来源 URL、命中方式与相似度）与脚本一并置于仓库的 `pipeline/` 与 `lit/` 目录。本文正文中的两个案例为自造教学实例，不含外部数据集。

参考文献

- [1] Yves Bertot and Pierre Castéran. *Interactive Theorem Proving and Program Development*. Springer Berlin Heidelberg, 2004. doi: 10.1007/978-3-662-07964-5. URL <https://doi.org/10.1007/978-3-662-07964-5>.
- [2] Stella Biderman, Hailey Schoelkopf, Lintang Sutawika, Leo Gao, Jonathan Tow, Baber Abbasi, Alham Fikri Aji, Pawan Sasanka Ammanamanchi, Sidney Black, Jordan Clive, Anthony DiPofi, Julen Etxaniz, Benjamin Fattori, Jessica Zosa Forde, Charles Foster, Jeffrey Hsu, Mimansa Jaiswal, Wilson Y. Lee, Haonan Li, Charles Lovering, Niklas Muennighoff, Ellie Pavlick, Jason Phang, Aviya Skowron, Samson Tan, Xiangru Tang, Kevin A. Wang, Genta Indra Winata, François Yvon, and Andy Zou. Lessons from the trenches on reproducible evaluation of language models, 2024. URL <https://arxiv.org/abs/2405.14782>. arXiv:2405.14782.
- [3] Wenhui Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks, 2022. URL <https://arxiv.org/abs/2211.12588>. arXiv:2211.12588.
- [4] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>. arXiv:2110.14168.
- [5] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models, 2022. URL <https://arxiv.org/abs/2211.10435>. arXiv:2211.10435.

- [6] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset, 2021. URL <https://arxiv.org/abs/2103.03874>. arXiv:2103.03874.
- [7] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet, 2023. URL <https://arxiv.org/abs/2310.01798>. arXiv:2310.01798.
- [8] Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs, 2022. URL <https://arxiv.org/abs/2210.12283>. arXiv:2210.12283.
- [9] Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Mudit Verma, Kaya Stechly, Siddhant Bhambri, Lucas Saldyt, and Anil Murthy. Llms can’t plan, but can help planning in llm-modulo frameworks. *Proceedings of the 41 st International Conference on Machine Learning, Vienna, Austria. PMLR 235, 2024*, 2024. URL <https://arxiv.org/abs/2402.01817>. arXiv:2402.01817.
- [10] Takeshi Kojima, Shixiang (Shane) Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems 35*, pages 22199–22213, 2022. doi: 10.52202/068431-1613. URL <https://arxiv.org/abs/2205.11916>. arXiv:2205.11916.
- [11] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step, 2023. URL <https://arxiv.org/abs/2305.20050>. arXiv:2305.20050.
- [12] The mathlib Community. The lean mathematical library. In *POPL ’20: 47th Annual ACM SIGPLAN Symposium on Principles of Programming Languages*, pages 367–381, 2020. doi: 10.1145/3372885.3373824. URL <https://doi.org/10.1145/3372885.3373824>.
- [13] Iman Mirzadeh, Keivan Alizadeh, Hooman Shahrokhi, Oncel Tuzel, Samy Bengio, and Mehrdad Farajtabar. Gsm-symbolic: Understanding the limitations of mathematical reasoning in large language models, 2024. URL <https://arxiv.org/abs/2410.05229>. arXiv:2410.05229.
- [14] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Lecture Notes in Computer Science*, pages 625–635, 2021. doi: 10.1007/978-3-030-79876-5_37. URL https://doi.org/10.1007/978-3-030-79876-5_37.
- [15] Yuansheng Ni, Sichao Jiang, Xinyu wu, Hui Shen, and Yuli Zhou. Evaluating the robustness to instructions of large language models, 2023. URL <https://arxiv.org/abs/2308.14306>. arXiv:2308.14306.

- [16] Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving, 2020. URL <https://arxiv.org/abs/2009.03393>. arXiv:2009.03393.
- [17] Oscar Sainz, Jon Campos, Iker García-Ferrero, Julen Etxaniz, Oier Lopez de Lacalle, and Eneko Agirre. Nlp evaluation in trouble: On the need to measure llm data contamination for each benchmark. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 10776–10787, 2023. doi: 10.18653/v1/2023.findings-emnlp.722. URL <https://arxiv.org/abs/2310.18018>. arXiv:2310.18018.
- [18] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36*, pages 68539–68551, 2023. doi: 10.52202/075280-2997. URL <https://arxiv.org/abs/2302.04761>. arXiv:2302.04761.
- [19] Kaya Stechly, Matthew Marquez, and Subbarao Kambhampati. Gpt-4 doesn’t know it’s wrong: An analysis of iterative prompting for reasoning problems, 2023. URL <https://arxiv.org/abs/2310.12397>. arXiv:2310.12397.
- [20] Christian Szegedy. A promising path towards autoformalization and general artificial intelligence. In *Lecture Notes in Computer Science*, pages 3–20, 2020. doi: 10.1007/978-3-030-53518-6_1. URL https://doi.org/10.1007/978-3-030-53518-6_1.
- [21] Amitayush Thakur, George Tsoukalas, Yeming Wen, Jimmy Xin, and Swarat Chaudhuri. An in-context learning agent for formal theorem-proving, 2023. URL <https://arxiv.org/abs/2310.04353>. arXiv:2310.04353.
- [22] Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024. doi: 10.1038/s41586-023-06747-5. URL <https://doi.org/10.1038/s41586-023-06747-5>.
- [23] Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback, 2022. URL <https://arxiv.org/abs/2211.14275>. arXiv:2211.14275.
- [24] Philip Wadler. Propositions as types. *Communications of the ACM*, 58(12):75–84, 2015. doi: 10.1145/2699407. URL <https://doi.org/10.1145/2699407>.
- [25] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35*, pages 24824–24837, 2022. doi: 10.52202/068431-1800. URL <https://arxiv.org/abs/2201.11903>. arXiv:2201.11903.

- [26] Yuhuai Wu, Albert Qiaochu Jiang, Wenda Li, Markus Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In *Advances in Neural Information Processing Systems 35*, pages 32353–32368, 2022. doi: 10.52202/068431-2344. URL <https://arxiv.org/abs/2205.12615>. arXiv:2205.12615.
- [27] Huajian Xin, Z. Z. Ren, Junxiao Song, Zhihong Shao, Wanjia Zhao, Haocheng Wang, Bo Liu, Liyue Zhang, Xuan Lu, Qiushi Du, Wenjun Gao, Qihao Zhu, Dejian Yang, Zhibin Gou, Z. F. Wu, Fuli Luo, and Chong Ruan. Deepseek-prover-v1.5: Harnessing proof assistant feedback for reinforcement learning and monte-carlo tree search, 2024. URL <https://arxiv.org/abs/2408.08152>. arXiv:2408.08152.
- [28] Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan J Prenger, and Animashree Anandkumar. Leandojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems 36*, pages 21573–21612, 2023. doi: 10.52202/075280-0944. URL <https://arxiv.org/abs/2306.15626>. arXiv:2306.15626.
- [29] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. Minif2f: a cross-system benchmark for formal olympiad-level mathematics, 2021. URL <https://arxiv.org/abs/2109.00110>. arXiv:2109.00110.
- [30] Aojun Zhou, Ke Wang, Zimu Lu, Weikang Shi, Sichun Luo, Zipeng Qin, Shaoqing Lu, Anya Jia, Linqi Song, Mingjie Zhan, and Hongsheng Li. Solving challenging math word problems using gpt-4 code interpreter with code-based self-verification, 2023. URL <https://arxiv.org/abs/2308.07921>. arXiv:2308.07921.
- [31] Jin Peng Zhou, Charles Staats, Wenda Li, Christian Szegedy, Kilian Q. Weinberger, and Yuhuai Wu. Don’t trust: Verify – grounding llm quantitative reasoning with autoformalization, 2024. URL <https://arxiv.org/abs/2403.18120>. arXiv:2403.18120.